

操作系统课程设计 2026(4, 进程通信 经典 IPC 问题 2 & 死锁)

计算机学院 邓蓉 2026/4/23

Part 1、嗜眠理发师问题

嗜眠理发师问题定义如下：

- 理发店有 N 名理发师， M 把椅子供顾客休息等待。没有顾客时，理发师睡觉。
- 顾客 FIFO 享受理发师提供的服务，对提供服务的理发师没有特殊要求。理发师全忙，坐在椅子上等；椅子坐满了，转身离开

我们一步一步来。先看一个简单的题。

一、快餐店问题



一、信号量的局限性

- **快餐店问题：**快餐店有 n 个座位。进入系统的用户看到座位全满、转身离开。

```
semaphore seat=n;
customer:
    p(seat);
    eating;
    v(seat);
    exit;
```

信号量的P操作	信号量的V操作
<pre>void p(S) { S.value -= 1; if(S.value < 0) sleep(); }</pre>	<pre>void v(S) { S.value += 1; if(S.value <= 0) wakeup(); }</pre>

错在哪里？

操作系统(拔尖班 / 国豪 2025)

同济大学计算机系 邓蓉

3

快餐店满的时候，新来的顾客无法离开，他阻塞等座位。

一个顾客一个位子，随手写出的解像这样，PPT3。

这个解错了，不满足题目要求，新顾客看到没有位子了，阻塞等待而不是立即离开。

我们改错。可以这样：用一个互斥信号量 mutex，管理系统状态的修改权限：

每次只允许一个顾客对系统状态进行观察、维护。



正确的解

```

semaphore mutex=1;
int seat=n;

customer进程
p(mutex);
if(seat==0)
{
    v(mutex);
    goto out;
}
else
    seat--;
v(mutex);

eating;
p(mutex);
seat++;
v(mutex);

out:
exit;

```

操作系统(拔尖班 / 国豪 2025) 同济大学计算机系 邓蓉 4

PPT4 是正解：新顾客持锁 mutex 看一眼快餐店顾客数量，如果是 0，转身离开。注意，修改系统状态后，无论离店还是落座就餐，不要忘记解锁 mutex。不解锁的话，系统会死掉。就餐完毕的顾客无法离店，新顾客进不来。

二、嗜眠理发师问题



正解

```

#define CHAIRS 5
semaphore customer=0;
semaphore barbers=0;
int waiting=0;
semaphore mutex=1;

void barber(void)
{
    While( TRUE ){
        P( customers );
        P( mutex );
        waiting = waiting-1;
        V( barbers );
        V( mutex );
        cuthair( );
    }
}

void customer(void)
{
    P(mutex);
    if( waiting < CHAIRS ){
        waiting = waiting+1;
        V( mutex );
        V( customers );
        P( barbers );
        get_haircut( );
    } else {
        V( mutex );
    }
}

```

操作系统(拔尖班 / 国豪 2025) 同济大学计算机系 邓蓉 9

信号量 customers，管理等待理发的顾客。没有顾客时，理发师 p(customers)睡觉，排在 customers.head 队列。

- 设置共享变量 waiting 同步 customer.value。理发店满的时候，新客户可以离开系统。
- 加互斥信号量 mutex，保护变量 waiting。

信号量 barbers，管理可以为用户提供服务的空闲理发师（睡眠理发师不算，因为唤醒后他才能为用户提供服务）。理发师全忙，顾客 P(barbers) 坐在长凳上等待。barbers.head 队列是长凳。

看 customer 进程。

P(mutex)互斥观察系统状态, L1、L2。

- 长凳满了, 离开系统, L9。
- 长凳不满, 执行进店操作。
 - 顾客数量++, 唤醒一个睡眠理发师。L3, L5。
 - 坐在长凳上等待理发, L6。
- 被选中的顾客, get_haircut()。

再看 barber 进程。

空闲下来后 (指 cuthair 函数返回, 或初始化阶段进入 while 循环), 执行 P(customers)同步等待客户 (会睡, 直到等到客户), 随后, 修正系统状态, 唤醒长凳队首客户, 为他服务 cuthair()。

P(mutex)确保每次只允许一个客户观察修正系统状态。注意每条分支都要有 V(mutex)。

本题为服务器线程池建模: 理发师是服务器端的线程池。负载过重客户可以离开。
这是个很好的系统。

Part 2、死锁



Part 3、死锁

- 在一个进程集合中, 每个进程都因等待永远不会发生的事件而阻塞, 称这种系统状态为死锁。
- 死锁产生的原因: 资源竞争, 资源分配策略不当 (进程推进顺序错误)。

▪ 引: 哲学家就餐问题

有五个哲学家公用一张圆桌。圆桌上有五个碗和五只筷子。哲学家思考, 饥饿时取用左边的筷子和右边的筷子。进餐完毕, 放下筷子继续思考。



会产生死锁的解

```
semaphore chopstick[5];
chopstick[0-4].value = 1;      /* 互斥信号量*/
void philosopher (int i)      /* i为执行此过程的哲学家号*/
{
    while (TRUE){
        think();              /* 思考问题，直至有饥饿感*/
        p( chopstick[i] );    /* 试图拿右侧筷子*/
        p( chopstick[(i+1)%N] ); /* 试图拿左侧筷子*/
        eat();                /* 两把叉子都得到就开始进餐*/
        v( chopstick[i] );    /* 放下右侧筷子*/
        v( chopstick[(i+1)%N] ); /* 放下左侧筷子*/
    }
}
```

- 5 位哲学家同时饥饿时， $p(\text{chopstick}[i])$ 。有可能会同时拿起右侧筷子。当它们继续前进、执行 $p(\text{chopstick}[i+1])$ 时，5 个进程均阻塞。
- 没有进程会执行 v 操作释放筷子的使用权（放下筷子）。系统状态不会改变了。
- 所有进程都在等待另一个进程释放资源给自己，死锁。



去除死锁，哲学家进餐问题的解

(1) 至多只允许有四位哲学家同时去拿左边的筷子，这样就能保证至少有一位哲学家能够进餐，并在用毕时能释放出他用过的两只筷子，从而使更多的哲学家能够用餐。。。鸽笼原理，变绝对的资源不足为相对的资源充沛。

(2) 仅当左、右两只筷子均可用时，哲学家才会申请筷子的使用权。

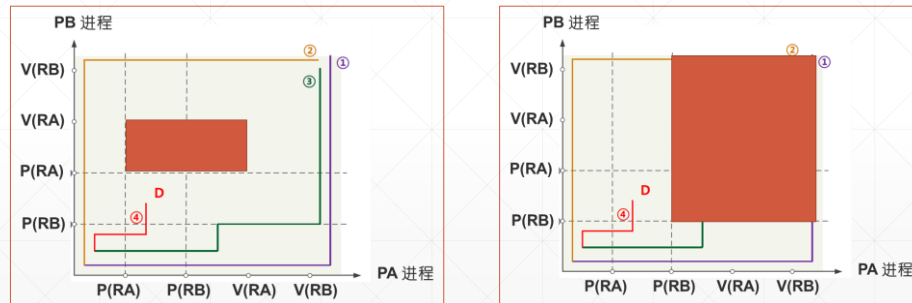
(3) 规定奇数号哲学家先拿他左边的筷子，然后再去拿右边的筷子，而偶数号哲学家则相反。按此规定，将是1、2号哲学家竞争1号筷子；3、4号哲学家竞争3号筷子。即五位哲学家都先竞争奇数号筷子，获得后，再去竞争偶数号筷子，最后总会有一位哲学家能够获得两只筷子而进餐。

下面我们研究去除死锁的策略。

首先，看会导致系统发生死锁的资源申请序列。



进程推进顺序不当会产生死锁



操作系统(拔尖班 / 国豪 2025)

同济大学计算机系 邓蓉

13

上图描述并发进程 PA、PB 进入临界区访问互斥资源 RA、RB 的情况。注：互斥资源不可以同时分配给 2 个以上的进程使用。硬件层面，所有外设都是互斥资源；软件层面，全局变量，只要有二个进程/线程同时访问的可能性，都是互斥资源。信号量、链表。。。全是。

(1) PA 进程执行代码：P(RA)，P(RB)，临界区 A，V(RA)，V(RB)。资源访问情况：先申请 RA 资源，再申请 RB 资源。获得 2 个资源的使用权后，进临界区对资源进行访问。结束后，释放 RA、RB 资源的使用权。

(2) PB 进程执行代码：P(RB)，P(RA)，临界区 B，V(RA)，V(RB)。它先申请 RB，再申请 RA 资源。。。

顺序执行上述所有操作，我们会得到一个线性轨迹。执行 PA 时，轨迹向右走。执行 PB 时，向上走。不存在其它轨迹延展方向。看例子，黑色的字是 PA 进程执行的操作，红色的字是 PB 进程。

左图，轨迹 1（轨迹先水平，再竖直向上走）：P(RA)，P(RB)，临界区 A，V(RA)，V(RB)，P(RB)，P(RA)，临界区 B，V(RA)，V(RB)。2 个资源先全部分配给 PA 用，释放后全部分配给 PB 使用。这条轨迹是合法的，因为不存在 1 个互斥资源同时分配给 2 个进程的情况。

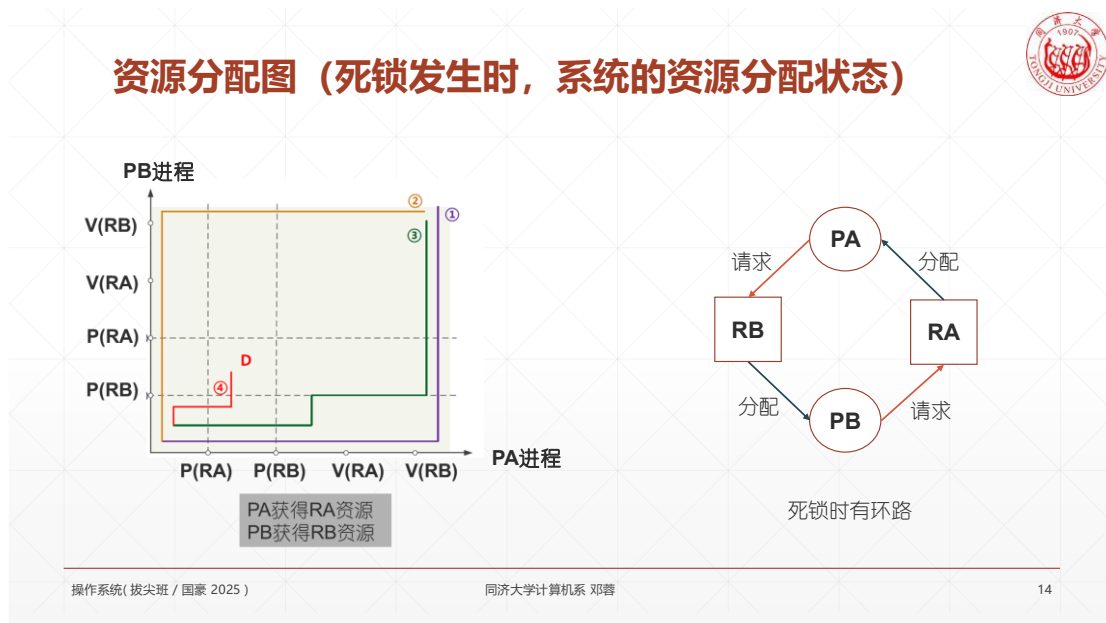
左图，轨迹 2（轨迹先竖直，再水平向右走）也是合法的。请同学们自行推导。

左图，轨迹 3（一条较为复杂的折线）：资源 RA、RB 分配给进程 PA 使用，临界区 A 执行一半的时候，PA 被剥夺（放弃 CPU 前依然持锁这两个资源）。。。PB 被选中执行（轨迹转向，水平变竖直向上走）。资源 RB 被 PA 锁住了，所以 PB 通不过 P(RB)，阻塞放弃 CPU。。。PA 恢复运行（轨迹转向，竖直变水平向右走），跑完临界区，释放资源，唤醒 PB。。。被唤醒的 PB（轨迹恢复向上走）通过 P(RB) 操作获得 RB 的使用权，拿到所有资源完成所有操作。没有违反互斥条件的时刻，轨迹 3 资源分配合法、安全。

看左图中的红色矩形区域。从左至右看，区域位于 PA 进程 P(RA) 和 V(RA) 操作之间，说明 RA 资源被 PA 进程持有；自下至上，区域位于 PB 进程 P(RA) 和 V(RA) 操作之间，RA 资源

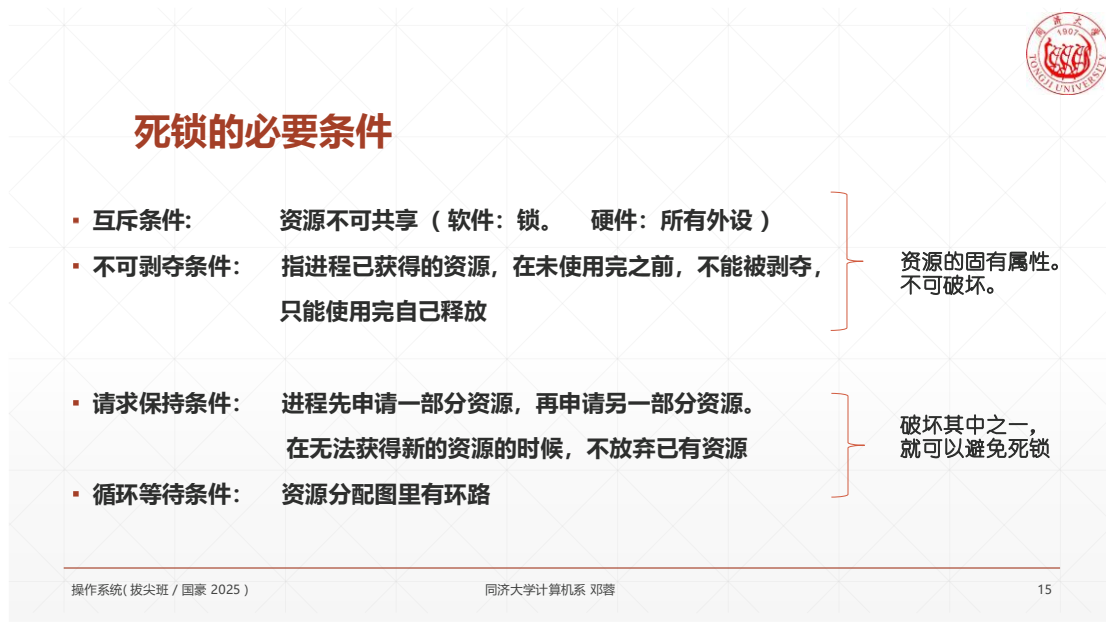
分配给 PB 进程。RA 是互斥资源，只能分配给一个进程；矛盾。这是轨迹无法进入的禁区。右图中的红色矩形区域是 RB 的禁区。同学们自行推导。

现在看区域 D，向上是 RA 的禁区，向右是 RB 的禁区。轨迹行至此已无路可走，死锁。对应的资源分配情况是：RA 资源分配给 PA 进程，RB 资源分配给 PB 进程。这两个进程均陷入等待另一个阻塞进程释放自己所需另一个资源的尴尬境地。资源分配图出现环路。



区域 D 是死锁区域，阻止轨迹进入该区域，系统就不会死锁。

死锁有 4 个必要条件。破坏其中任何一个，死锁就不会发生。



换一种说法。

死锁，一定是资源分配不当。

死锁涉及的所有资源一定是互斥的。释放资源的操作只能由持锁进程执行。

进程先申请一部分资源，再申请一部分资源，才会产生死锁。换言之，为进程一次性分配所有资源，就不会有死锁。

死锁发生时，资源请求图里有环路。

应对死锁有 3 种措施：(1) 设计系统时，认真编码，破坏死锁必要条件 3 或 4，预防并发系统出现死锁 (2) 系统设置集中式资源分配元件评估进程提出的资源申请，拒绝一切可能会导致系统进入不安全状态的资源访问请求，避免并发系统发生死锁现象。请求被拒后，进程阻塞等待可用资源 (3) 前两种措施代价高昂，会降低资源使用效率和系统并发度。所以，很多系统，资源分配时不防范死锁，后台定期执行死锁检测工具，识别资源分配环路和陷入死锁的进程。用杀死进程的方法解除死锁。

处理死锁的方法



- **预防死锁** 编码时破坏死锁产生的必要条件
 - 一次性给进程分配所需的全部资源 破坏请求保持条件
 - 保证资源分配图不出现环路 破坏循环等待条件
- **避免死锁** 运行时实施。评估进程提出的资源使用申请，避免系统陷入不安全状态。

系统不会发生死锁
(代价高昂)

- **检测死锁 & 解除死锁**
 - 为了提高系统的效率，不防范死锁 (鸵鸟政策)
 - 定期执行检测工具，检测资源分配环路 和 陷入死锁的进程。
 - 逐个杀死重启进程，直至恢复正常运行状态。

下面以哲学家进餐问题为例介绍编程预防死锁的方法。

预防死锁方法一：破坏请求保持条件，一次性为进程分配所有资源。有两种实现方式，1、系统提供同步信号量原语 p，进程用它同时申请多个资源的使用权限，拿到所有资源的访问权限后 (本题 2 只筷子)，p 操作才会返回。

破坏请求保持条件



- 使用**同步信号量**，进程一次性申请和释放运行所需的全部资源。
- 第一种解

```
semaphore chopstick[5];
chopstick[0-4].value = 1;
void philosopher (int i)
{
    while (TRUE)
    {
        think();
        p( chopstick[i], chopstick[(i+1)%N] );    //if system support this tool😂
        eat();
        v( chopstick[i], chopstick[(i+1)%N] );
    }
}
```


2、用 P、V 操作仔细编程，一次性为进程分配全部资源。让我们借鉴一个优秀的设计。

第二种解



```
#define N 5
#define LEFT (i+N-1)%N
#define RIGHT (i+1)%N
#define THINKING 0
#define HUNGRY 1
#define EATING 2
int state[N];
semaphore mutex=1;
semaphore s[N]={0,0,0,0,0};

void philosopher(int i)
{
    while(TRUE){
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}

void take_forks(int i)
{
    p(mutex);
    state[i]=HUNGRY;
    test(i);
    v(mutex);
    p(s[i]);
}

void put_forks(int i)
{
    p(mutex);
    state[i]=THINKING;
    test(LEFT);
    test(RIGHT);
    v(mutex);
}

void test(i)
{
    if( state[i] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING)
    {
        state[i]=EATING;
        v(s[i]);
    }
}
```

操作系统(拔尖班 / 国豪 2025)

同济大学计算机系 邓蓉

18

state[i]是第 i 个哲学家的状态：思考、饥饿，进餐。

资源信号量 s[i]管理哲学家的进餐权限。

互斥信号量 mutex，确保所有时刻，至多一个哲学家可以执行资源分配、释放操作。这个哲学家可能是：

- 饥饿的哲学家。就餐前，为自己检测、分配筷子。
- 进餐结束的哲学家。进餐后（1）放下筷子（2）为左边和右边的哲学家分配筷子。

分配原则是，要就 2 只，否则一只也不要碰。是这样的：

饥饿 & 拿到 2 只筷子的哲学家，可以通过 P(s[i])。

饥饿 & 没拿到 2 只筷子的哲学家，P(s[i])阻塞。

这是耳目一新的思路。

看代码，涉及资源管理的是哲学家的 2 个动作，

- (1) take_forks(i)，i#哲学家申请就餐权限。方法如下：

哲学家 p(mutex)获得资源分配权限。

他调用 test()函数看一下筷子的分配情况，如果满足要求，v(s[i])为自己发放就餐许可证。test()函数返回后，哲学家会执行 p(s[i])操作，不具备就餐条件阻塞等待。可以就餐时用掉刚刚获得的就餐许可证。

这是一次性分配所有资源的策略。它破坏了循环等待条件，系统不可能死锁。

- (2) put_forks(i)，i#哲学家进餐完毕后，放下筷子，test(LEFT)和 test(RIGHT)为左边的哲学家和右边的哲学家发放就餐许可证。同学们自行探索。

预防死锁方法二：破坏循环等待条件。方法是：设计系统时，为资源编号。所有进程，获得编号小的资源后，才能够获得编号大的资源。可以逐步申请资源，资源申请只能沿一个方向，不可能有环，对吧。看代码，所有哲学家先拿小号筷子，再拿大号筷子。



破坏循环等待条件

- 为资源编号，强制要求：进程只有在获得编号小的资源之后才可以获得编号大的资源。

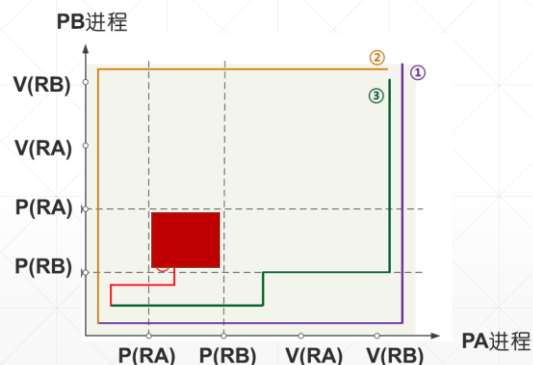
```
void philosopher(int i)
{
    while(TRUE)
    {
        think( );
        if( i==4) { P(fork[0]); P(fork[4]); }
        else { P(fork[i]); P(fork[i+1]); }
        eat( );
        if(i==4) { V(fork[0]); V(fork[4]); }
        else { V(fork[i]); V(fork[i+1]); }
    }
}
```

下面介绍银行家算法：该算法拒绝可能导致轨迹进入红色区域的资源分配请求。



死锁预防策略 银行家算法

- 系统在进行资源分配之前，应先计算此次资源分配的安全性。若此次分配不会导致系统进入不安全状态（红色区域），则将资源分配给进程；否则，令进程等待。
- 只要能使系统始终都处于安全状态，便可避免发生死锁。





安全状态

- **安全状态**是系统的一种资源分配状态。在这种状态下，存在**安全序列** (P1, P2, ..., Pn)。系统可以按进程在安全序列中出现的次序依次为它们分配运行所需的全部资源，直至所有进程能够顺利完成。

例：



- 假定系统中有三个进程P1、P2 和 P3，共有12台磁带机。进程P1总共要求10台磁带机，P2和P3分别要求4台和9台。假设在T0时刻，进程P1、P2和P3已分别获得5台、2台和2台磁带机，尚有3台空闲未分配，如表2.3所示。

1、系统安全吗？

存在安全序列
<P2, P1, P3>

安全状态

进程	最大需求	已分配	可用
P1	10	5	3
P2	4	2	
P3	9	2	

为进程 P2 分配 2 台磁带机，可以满足它的资源最大需求。

P2 运行完毕后，释放所有资源，系统可用资源总数上升至 5。

将这 5 台打印机全部分给 P1 进程，满足它的资源最大需求。

P1 运行完毕后，释放所有资源，系统可用资源总数上升至 10。

10 台打印机，满足 P3 的资源最大需求。P3 也可以顺利运行完毕。

表格所示资源分配状态安全，进程按安全序列<P2、P1、P3>申请资源可以全部执行完毕。

让我们研究不安全的资源分配策略：

例：

- 假定系统中有三个进程P1、P2和P3，共有12台磁带机。进程P1总共要求10台磁带机，P2和P3分别要求4台和9台。假设在T0时刻，进程P1、P2和P3已分别获得5台、2台和2台磁带机，尚有3台空闲未分配，如表2.3所示。

2、P3再申请1台磁带机。系统可以满足这次资源申请请求吗？

进程	最大需求	已分配	可用
P1	10	5	3 2
P2	4	2	
P3	9	2 3	

2台磁带机全部分配给P2进程。P2进程可以运行结束。之后，可用磁带机的数量增至4台。

这4个资源无法满足P1进程的需求，也无法满足P3进程的需求。P3进程的这次资源申请请求会使系统进入不安全状态。系统不能满足此次资源申请，应该让P3进程入睡等待。

评估放款安全性，确保银行资金安全。借鉴银行家批贷款的思路，银行家算法。

银行家算法的实现：用来描述资源分配状态的数据结构

- n个进程，m种资源
- 空闲资源向量 Available，m维：
Available[j] 是空闲资源 j 的数量。
- 最大资源需求矩阵 Max，n*m维：
Max[i][j] 是进程 Pi 对资源 j 的最大需求量。
- 资源分配矩阵 Allocation，n*m维：
Allocation[i][j] 是已经分配给进程 Pi 的资源 j 的数量。
- 资源缺口矩阵 Need，n*m维：Need = Max - Allocation
为进程 Pi 追加资源 Need[i]，可顺利运行结束。

Available > Need[i] Pi进程可以运行结束

所有Need[i]>available 不安全状态

所有 Need[i]>available，空闲资源无法支撑任何进程运行至结束。资源分配不安全状态。

安全状态检测算法

银行家算法的实现：用来描述资源分配状态的数据结构

- n 个进程， m 种资源
- 空闲资源向量 Available, m 维：
Available[j] 是空闲资源 j 的数量。
- 最大资源需求矩阵 Max, $n \times m$ 维：
Max[i][j] 是进程 P_i 对资源 j 的最大需求量。
- 资源分配矩阵 Allocation, $n \times m$ 维：
Allocation[i][j] 是已经分配给进程 P_i 的资源 j 的数量。
- 资源缺口矩阵 Need, $n \times m$ 维：Need = Max - Allocation
为进程 P_i 追加资源 Need[i]，可顺利运行结束。

Available > Need[i] P_i 进程可以运行结束 所有Need[i]>available 不安全状态

finish, n 维布尔向量。每个进程一个元素。

if(Available > Need[i])

finish[i]=true; //进程 P_i 能够获得所需全部资源运行结束。

```
Safety()
{
    finish[n] = {false, false, ..., false};
    work = Available;
    while(finish向量存在false成员)
    {
        if(存在 need[i] <= work) {
            work += allocation[i];
            finish[i] = true;
        }
        else
            return( false )
    }
    return( true )
}
```

寻找可以顺利运行结束的进程，满足它的资源需求。

运行结束后，这个进程会释放资源。空闲资源数量会变多。

再找一个这样的进程。。。

如果最后，所有进程都可以运行结束。原先的资源分配状态安全。

银行家算法

P_i 进程

```
{
    .....
    ResourceRequest (i, Request);
    占用并使用资源;
    .....
    释放资源;
    .....
}
```

```
ResourceRequest( i, Request )
{
    if ( Request > Need[i] )
        exit();
    if ( Request > Available )
        sleep();
    while( ! Alloc_Banker( i, Request ) )
        sleep();
    return;
}
```

进程 P_i 发出资源请求是 m 维的Request向量
Request[j]是资源 j 的需求量

```
Alloc_Banker( i, Request ) {
    P( mutex );
    Available -= Request;
    Allocation[i] += Request;
    Need[i] -= Request;
    if ( Safety() )
    {
        V(mutex );
        return( true );
    }
    else
    {
        Available += Request;
        Allocation[i] -= Request;
        Need[i] += Request;
        V(mutex );
        return( false );
    }
}
```

P_i 进程按需申请资源，ResourceRequest (i, Request)。Request 是其申请的资源向量。

- (1) 进程使用资源超过
- (2) 资源分配给它，系统会进入不安全状态。 P_i 睡眠等待。
- (3) 资源分配安全，ResourceRequest()函数返回。

Alloc_Banker(i, Request), 银行家算法。评估资源分配后系统的安全性。安全，返回 true。否则，返回 false。注意，算法在数据结构层面运算，没有动 P 操作，没有实质性的资源分配。所以，发现资源分配不安全，数据结构复原即可。

ResourceRequest()函数返回后，一次性为进程分配所有资源。这是实质性的分配资源，资源分配已经过安全性评估，不会死锁，安全。

计算题，自己来吧。

银行家算法的例子

系统中有五个进程 {P0,P1,P2,P3,P4} 和三类资源{A,B,C}，资源总数为10,5,7，可用资源为3,3,2。请问T0时刻，系统状态安全嘛？

进程	Max A B C	Allocation A B C	Need A B C
P0	7 5 3	0 1 0	7 4 3
P1	3 2 2	2 0 0	1 2 2
P2	9 0 2	3 0 2	6 0 0
P3	2 2 2	2 1 1	0 1 1
P4	4 3 3	0 0 2	4 3 1

Work==Available==[3,3,2]

进程	Max A B C	Allocation A B C	Need A B C	work A B C
P0	7 5 3	0 1 0	7 4 3	
P1	3 2 2	2 0 0	1 2 2	1: 5 3 2
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	2: 7 4 3
P4	4 3 3	0 0 2	4 3 1	

存在安全序列
<P1、P3、P0、P2、P4>
<P1、P3、P2、P4、P0>
<P1、P3、.....>

操作系统(拔尖班 / 国豪 2025)同济大学计算机系 邓蓉27

银行家算法的例子（续）

系统中有五个进程 {P0,P1,P2,P3,P4} 和三类资源{A,B,C}，资源总数为10,5,7，可用资源为3,3,2。P0发出请求向量Request0 = (3, 3, 0)，系统可以满足这个进程的资源申请嘛？

满足P1的资源申请后：Work==Available==[0,0,2]，无法满足任何进程资源访问请求，不安全
所以系统应该拒绝P0的资源申请。

进程	Max A B C	Allocation A B C	Need A B C
P0	7 5 3	0 1 0	7 4 3
P1	3 2 2	2 0 0	1 2 2
P2	9 0 2	3 0 2	6 0 0
P3	2 2 2	2 1 1	0 1 1
P4	4 3 3	0 0 2	4 3 1

进程	Max A B C	Allocation A B C	Need A B C
P0	7 5 3	3 4 0	4 1 3
P1	3 2 2	2 0 0	1 2 2
P2	9 0 2	3 0 2	6 0 0
P3	2 2 2	2 1 1	0 1 1
P4	4 3 3	0 0 2	4 3 1

操作系统(拔尖班 / 国豪 2025)同济大学计算机系 邓蓉28

评论：银行家算法要求 Alloc_Banker()知道所有进程资源分配需求，就是那个 Max 矩阵。如果进程执行前给不出这个信息，银行家算法就没法儿跑了。

所以，教科书上说这是理论上的资源分配算法，实际上实现不了。